2025년 홍익대학교 AI 기술 트렌드 분석 및 신규 AI 서비스 프로젝트 제안서

1. 개요

본 문서는 2025년 홍익대학교 컴퓨터공학과 졸업 프로젝트 기술 백서와 최신 생성형 AI 트렌드를 분석하여, **학술적 깊이와 실용성을 겸비한** 신규 졸업 프로젝트 아이디어 **10개**를 제안합니다.

1.1 제안 조건

- **기술 스택**: Express + Node.js + MongoDB (+ Python/LLM API 연동)
- **지도교수 적합성**: 운영체제 전공 교수님 → 시스템 관점에서의 AI 서빙, 리소스 관리, 스케줄링 개념 융합
- **난이도**: 1인 개발 가능하면서도 학술적 의미가 있는 수준
- **차별화**: 단순 API 호출이 아닌, OS/시스템 이론을 AI 도메인에 적용

1.2 2025년 생성형 AI 핵심 트렌드

> **"2025년은 생성형 AI가 실험 단계를 벗어나 실전 비즈니스 가치를 창출하는 해"**

트렌드	설명	학술적 연구 포인트
🤖 **Agentic AI**	스스로 계획·실행·최적화하는 자율형 AI	에이전트 아키텍처, 멀티 에이전트 협업
📚 **RAG**	외부 지식을 검색하여 LLM 정확도 향상	검색 알고리즘 비교, 하이브리드 검색
⚡ **LLM 서빙 최적화**	추론 비용/지연 시간 최적화	스케줄링, 캐싱, 배칭 알고리즘
🔒 **온디바이스 AI**	경량 모델 + 개인 정보 보호	엣지 컴퓨팅, 리소스 제약 환경
✅ **AI 신뢰성**	환각 탐지, 팩트 체크, 출처 추적	NLI, 자동 검증 파이프라인

---

2. 프로젝트 아이디어 제안 (10개)

📊 전체 비교표

#	프로젝트명	OS 연관성	AI 깊이	실용성	난이도
1	LLM 추론 요청 스케줄러	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐	중상
2	멀티 에이전트 오케스트레이터	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	상
3	에이전트 메모리 관리 시스템	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐	중상
4	에이전트 샌드박스 실행 환경	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐⭐	중상
5	자율형 코드 디버깅 에이전트	⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	상
6	에이전트 워크플로우 컴파일러	⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐	중상
7	에이전트 데드락 탐지 시스템	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐	상
8	분산 에이전트 합의 프로토콜	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐	상
9	에이전트 체크포인팅 시스템	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐	중
10	자기 진화형 프롬프트 에이전트	⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	중상

1️⃣ LLM 추론 요청 스케줄러 ⭐ 운영체제 교수 최적 추천

문제 정의

- 다수 사용자가 LLM API를 동시 호출 시 비용 폭증 + 응답 지연
- 모든 요청을 동일하게 처리하면 중요 요청도 대기

해결 방안

OS 프로세스 스케줄링 알고리즘(MLFQ, Priority Queue, Weighted Fair Queue)을 LLM 요청에 적용 - 우선순위, 토큰 예산, 대기시간 기반 동적 스케줄링 - 요청 배칭(batching)으로 처리량 최적화

핵심 기술

```
Express Proxy Server, 요청 큐잉 (Bull/BullMQ)
우선순위 알고리즘 구현, MongoDB (요청 이력/통계)
LLM API 연동, 실시간 대시보드 (Socket.io)
```

학술적 가치

- **OS 스케줄링 이론**을 신규 도메인(LLM 서빙)에 적용
- 알고리즘별 성능 비교 실험 (처리량, 평균 대기시간, 공정성)
- 논문 작성 가능: “LLM 추론 요청을 위한 적응형 스케줄링 알고리즘”

예상 개발 기간

- MVP: 5주 / 완성본: 10주

2️⃣ 멀티 에이전트 오케스트레이터 ⭐ OS 프로세스 관리 적용

문제 정의

- 복잡한 작업을 단일 LLM 에이전트로 처리하면 성능/품질 한계
- 여러 전문 에이전트가 협업해야 하지만, 동기화/통신/자원 경쟁 문제 발생

해결 방안

OS의 프로세스 관리 개념을 멀티 에이전트 시스템에 적용 - **IPC(Inter-Process Communication)** 패턴으로 에이전트 간 메시지 전달 - **Semaphore/Mutex**로 공유 자원(Context Window, API Rate Limit) 관리 - **Fork-Join 모델**로 병렬 작업 분배 및 결과 취합

핵심 기술

```
에이전트 프로세스 모델 설계
메시지 큐 (Redis Pub/Sub, BullMQ)
공유 상태 관리 (MongoDB + 락 메커니즘)
Express 오케스트레이션 서버
LangGraph / AutoGen 참조 아키텍처
```

학술적 가치

- **OS IPC 이론**을 AI 에이전트 도메인에 적용
- 멀티 에이전트 협업 패턴 분류 및 성능 비교
- 논문 작성 가능: "프로세스 간 통신 모델 기반 멀티 AI 에이전트 오케스트레이션"

예상 개발 기간

- MVP: 6주 / 완성본: 12주

3️⃣ 에이전트 메모리 관리 시스템 ⭐ OS 메모리 관리 적용

문제 정의

- LLM 에이전트의 Context Window는 제한적 (128K 토큰 등)
- 긴 대화/복잡한 작업 시 중요 정보가 밀려나 "망각" 발생
- 무엇을 기억하고 무엇을 잊을지 관리 필요

해결 방안

OS 메모리 관리 기법을 에이전트 컨텍스트 관리에 적용 - **페이징**: 컨텍스트를 "페이지" 단위로 분할, 필요 시 스왑 - **페이지 교체 알고리즘**: LRU, LFU, Working Set 모델로 중요 정보 유지 - **메모리 계층**: L1(Context Window) → L2(Vector Store) → L3(Persistent DB)

핵심 기술

```
컨텍스트 청킹 및 중요도 점수 산출
벡터 DB (MongoDB Atlas Search, Pinecone)
페이지 교체 알고리즘 구현
Express, LLM API
실시간 메모리 사용량 시각화
```

학술적 가치

- **OS 가상 메모리/페이징 이론**의 LLM 적용

- 에이전트 장기 기억 관리 연구
- 논문 작성 가능: "LLM 에이전트를 위한 계층적 메모리 관리 시스템"

예상 개발 기간

- MVP: 5주 / 완성본: 10주

4️⃣ 에이전트 샌드박스 실행 환경 ⭐ OS 보안/격리 적용

문제 정의

- AI 에이전트가 코드 실행, 파일 접근 등 위험한 작업 수행 시 보안 위협
- 악의적 프롬프트로 시스템 탈취 가능성
- 신뢰할 수 없는 에이전트 실행의 안전한 격리 필요

해결 방안

OS의 Sandboxing/Containerization 개념을 에이전트 실행에 적용 - **권한 분리**: 에이전트별 Capability 기반 접근 제어 - **리소스 제한**: CPU/메모리/네트워크/API 호출 쿼터 - **격리 실행**: Docker/Firecracker 기반 코드 실행 샌드박스

핵심 기술

```
Docker SDK / Firecracker microVM
Capability 기반 권한 모델 설계
리소스 쿼터 및 모니터링 (cgroups 개념)
Express 오케스트레이션, MongoDB (감사 로그)
실시간 에이전트 활동 모니터링 대시보드
```

학술적 가치

- **OS 보안/격리 이론**을 AI 에이전트에 적용
- AI 에이전트 보안 프레임워크 설계
- 논문 작성 가능: "신뢰할 수 없는 AI 에이전트의 안전한 실행을 위한 샌드박스 아키텍처"

예상 개발 기간

- MVP: 5주 / 완성본: 10주

5️⃣ 자율형 코드 디버깅 에이전트

문제 정의

- 버그 수정은 시간이 많이 드는 반복 작업
- 에러 로그 분석 → 원인 파악 → 수정 → 테스트의 사이클이 길고 지루함
- 기존 도구는 "제안"만 하고 실제 수정/검증은 개발자 몫

해결 방안

버그 수정 전 과정을 자동화하는 자율형 에이전트 구축 - **Observe**: 에러 로그, 스택 트레이스, 테스트 실패 분석 - **Orient**: 원인 가설 생성 (LLM 추론) - **Decide**: 수정 방안 선택 - **Act**: 코드 패치 생성 → 테스트 실행 → 결과 피드백 루프

핵심 기술

```
AST 조작 (Tree-sitter, Babel)
테스트 러너 연동 (Jest, pytest)
LLM 기반 코드 생성/수정
```

```
Git 연동 (자동 브랜치/커밋)
Express, MongoDB (디버깅 세션 기록)
```

학술적 가치

- **OODA 루프** 기반 자율 에이전트 설계
- 자동 프로그램 수정(APR) 분야 기여
- 논문 작성 가능: "피드백 루프 기반 자율형 코드 디버깅 에이전트"

예상 개발 기간

- MVP: 6주 / 완성본: 11주

6️⃣ 에이전트 워크플로우 컴파일러

문제 정의

- 에이전트 워크플로우를 자연어로 정의하면 실행 효율이 낮음
- 매번 LLM이 다음 단계를 추론 → 느리고 비용 높음
- 반복적인 워크플로우를 최적화할 방법 필요

해결 방안

자연어 워크플로우를 분석하여 최적화된 실행 계획으로 "컴파일" - **Parser**: 자연어 → AST(Abstract Syntax Tree) - **Optimizer**: 병렬화 가능 단계 탐지, 중복 제거, 캐싱 적용 - **Code Generator**: 최적화된 실행 코드/DAG 생성 - **Runtime**: 컴파일된 워크플로우 실행

핵심 기술

```
자연어 → 구조화된 플랜 변환 (LLM)
DAG(Directed Acyclic Graph) 생성 및 최적화
실행 엔진 (병렬 처리, 조건 분기)
Express, MongoDB (워크플로우 저장)
시각적 워크플로우 에디터
```

학술적 가치

- **컴파일러 이론**(렉싱, 파싱, 최적화)을 에이전트 도메인에 적용
- 에이전트 실행 효율 정량적 개선 측정
- 논문 작성 가능: "자연어 에이전트 워크플로우의 컴파일 및 최적화"

예상 개발 기간

- MVP: 5주 / 완성본: 10주

7️⃣ 에이전트 데드락 탐지 및 해결 시스템 ⭐ OS 교수 적합

문제 정의

- 멀티 에이전트 시스템에서 상호 대기(Circular Wait) 발생 가능
- 예: 에이전트 A가 B의 출력을 기다리고, B가 A의 출력을 기다림
- 무한 대기로 시스템 전체 멈춤

해결 방안

OS 데드락 이론을 멀티 에이전트 시스템에 적용 - **탐지**: Wait-For Graph 구축 → 사이클 탐지 알고리즘 - **예방**: Banker's Algorithm 변형으로 자원 할당 사전 검사 - **회복**: 피해 최소 에이전트 선택 → 롤백 또는 강제 종료

핵심 기술

```
Wait-For Graph 구현 및 사이클 탐지
에이전트 상태 스냅샷 및 롤백 메커니즘
실시간 모니터링 대시보드
Express, MongoDB, Socket.io
```

학술적 가치

- **OS 데드락 이론**의 AI 에이전트 적용
- 멀티 에이전트 신뢰성/안정성 연구
- 논문 작성 가능: "멀티 AI 에이전트 시스템의 데드락 탐지 및 회복 알고리즘"

예상 개발 기간

- MVP: 5주 / 완성본: 9주

8️⃣ 분산 에이전트 합의 프로토콜 ⭐ OS/분산시스템 적용

문제 정의

- 여러 에이전트가 동일 목표를 위해 협업할 때, 의사결정 충돌 발생
- 예: 코드 리뷰 에이전트들이 서로 다른 수정안을 제시
- 일관된 최종 결정을 도출하는 합의 메커니즘 필요

해결 방안

분산 시스템의 합의 프로토콜을 에이전트 협업에 적용 - **투표 기반**: 다수결, 가중 투표 (에이전트 전문성 기반) - **Raft 변형**: 리더 선출 → 리더가 최종 결정 - **Byzantine Fault Tolerance 경량화**: 악의적/비정상 에이전트 대응

핵심 기술

```
합의 프로토콜 구현 (Raft 변형, PBFT 경량화)
에이전트 신뢰도 점수 시스템
결정 이력 및 감사 로그
Express, MongoDB, Redis (상태 동기화)
```

학술적 가치

- **분산 시스템 합의 이론**의 AI 에이전트 적용
- 멀티 에이전트 의사결정 품질 연구
- 논문 작성 가능: "다중 AI 에이전트 시스템을 위한 합의 프로토콜 설계"

예상 개발 기간

- MVP: 5주 / 완성본: 10주

9️⃣ 에이전트 체크포인팅 시스템 ⭐ OS 프로세스 마이그레이션 적용

문제 정의

- 장시간 실행되는 에이전트 작업 중 장애 발생 시 처음부터 재시작
- 비용과 시간 낭비, 중간 결과 손실
- 에이전트 상태를 저장하고 복원하는 메커니즘 필요

해결 방안

OS의 프로세스 체크포인팅/마이그레이션 개념을 에이전트에 적용 - **스냅샷**: 에이전트 상태(컨텍스트, 변수, 진행도) 직렬화 - **복원**: 스냅샷에서 에이전트 재개 - **마이그레이션**: 다른 서버/환경으로 에이전트 이동

핵심 기술

```
에이전트 상태 직렬화/역직렬화
증분 체크포인팅 (변경분만 저장)
MongoDB (스냅샷 저장), S3 (대용량 상태)
Express, 상태 복원 검증 시스템
```

학술적 가치

- **OS 체크포인팅/CRIU 개념**의 AI 에이전트 적용
- 장기 실행 에이전트의 신뢰성 향상 연구
- 논문 작성 가능: "장기 실행 AI 에이전트를 위한 체크포인팅 및 복구 시스템"

예상 개발 기간

- MVP: 4주 / 완성본: 8주

🔟 자기 진화형 프롬프트 에이전트 (Self-Improving Agent)

문제 정의

- 에이전트의 성능은 프롬프트 품질에 크게 의존
- 최적의 프롬프트를 찾기 위해 수작업 시행착오 필요
- 도메인/작업이 바뀌면 프롬프트 재작성 필요

해결 방안

에이전트가 자신의 프롬프트를 자동으로 개선하는 메타 학습 시스템 - **실행**: 현재 프롬프트로 작업 수행 - **평가**: 결과 품질 자동 측정 (정확도, 사용자 피드백) - **진화**: 프롬프트 변이(Mutation) → 선택(Selection) → 다음 세대 - **유전 알고리즘** 또는 **강화학습** 기반 최적화

핵심 기술

```
프롬프트 변이 전략 (LLM 기반 리라이팅)
자동 평가 메트릭 설계
유전 알고리즘 / Bayesian Optimization
Express, MongoDB (프롬프트 버전 관리)
A/B 테스트 프레임워크
```

학술적 가치

- **진화 알고리즘** / **AutoML** 개념의 프롬프트 최적화 적용
- 프롬프트 엔지니어링 자동화 연구
- 논문 작성 가능: "유전 알고리즘 기반 자기 진화형 LLM 프롬프트 최적화"

예상 개발 기간

- MVP: 5주 / 완성본: 10주

---

3. 프로젝트 추천

3.1 운영체제 교수님 추천 TOP 3

순위	프로젝트	추천 이유
🥇	**LLM 추론 요청 스케줄러 (#1)**	OS 스케줄링 알고리즘(MLFQ, Priority)을 직접 구현하고 LLM 도메인에 적용. 가장 명확한 OS 연관성
🥈	**에이전트 데드락 탐지 시스템 (#7)**	OS 데드락 이론(Wait-For Graph, Banker's Algorithm)을 멀티 에이전트에 적용. 클래식 OS 개념
🥉	**에이전트 메모리 관리 시스템 (#3)**	페이징, LRU/LFU 페이지 교체를 에이전트 컨텍스트에 적용. 가상 메모리 이론 활용

3.2 학술적 가치 높은 프로젝트 TOP 3

순위	프로젝트	추천 이유
🥇	**멀티 에이전트 오케스트레이터 (#2)**	IPC, 동기화, Fork-Join 등 OS 이론 + 최신 Agentic AI 트렌드 융합. 논문 주제로 최적
🥈	**분산 에이전트 합의 프로토콜 (#8)**	Raft, PBFT 등 분산시스템 이론을 AI 에이전트에 적용. 학술적 독창성 높음
🥉	**자기 진화형 프롬프트 에이전트 (#10)**	유전 알고리즘 + AutoML + LLM 융합. 메타 학습 연구 분야 기여

3.3 실용성 높은 프로젝트 TOP 3

순위	프로젝트	추천 이유
🥇	**자율형 코드 디버깅 에이전트 (#5)**	개발자 생산성 직접 향상. GitHub Copilot 수준을 넘어선 자율 수정
🥈	**에이전트 샌드박스 실행 환경 (#4)**	AI 에이전트 보안 문제 해결. 기업 도입 시 필수 요소
🥉	**에이전트 체크포인팅 시스템 (#9)**	장기 실행 에이전트의 안정성 보장. 실제 프로덕션 환경 필수 기능

---

4. 상위 프로젝트 상세 명세

4.1 LLM 추론 요청 스케줄러

아키텍처

```
flowchart LR
    A[클라이언트 요청] --> B[게이트웨이<br/>Express Proxy]
    B --> C[요청 분류기]
    C -->|우선순위/예산/요청 유형 분석| D[스케줄러]
    D -->|MLFQ / Priority / Fair Queue| E[요청 큐]
    E -->|동적 우선순위 조정| F[디스패처]
```

```
    F -->|배칭, Rate Limit| G[LLM API 호출]
    G --> H[응답 반환 +<br/>메트릭 수집]
```

구현할 스케줄링 알고리즘

알고리즘	설명	적용 시나리오
FCFS	선착순 처리	기본 베이스라인
Priority Queue	우선순위별 처리	VIP 사용자 우선
MLFQ	다단계 피드백 큐	혼합 워크로드
Weighted Fair Queue	가중치 기반 공정 분배	멀티테넌트 환경
Token Budget Scheduler	토큰 예산 기반	비용 최적화

성능 측정 지표

- 평균 대기 시간 (Average Wait Time)
- 처리량 (Throughput)
- 공정성 지수 (Fairness Index)
- 비용 효율 (Cost per Request)

예상 개발 기간

단계	기간	산출물
설계	1주	아키텍처 문서, API 설계
MVP	4주	기본 스케줄러 (FCFS, Priority)
고도화	3주	MLFQ, WFQ 구현
실험	2주	알고리즘별 성능 비교 실험

4.2 멀티 에이전트 오케스트레이터

아키텍처

```
flowchart LR
    A[사용자 태스크 입력] --> B[태스크 분해기<br/>Task Decomposer]
    B --> C[에이전트 스케줄러]
    C -->|Fork-Join| D1[Agent A<br/>검색]
    C -->|Fork-Join| D2[Agent B<br/>코드]
    C -->|Fork-Join| D3[Agent C<br/>분석]
    C -->|Fork-Join| D4[Agent D<br/>작성]
    D1 --> E[IPC 메시지 큐]
    D2 --> E
    D3 --> E
    D4 --> E
    E -->|Semaphore/Mutex 동기화| F[결과 취합기<br/>Aggregator]
    F --> G[최종 응답]
```

구현할 IPC 패턴

패턴	설명	적용 시나리오
Pipe	단방향 순차 전달	파이프라인 처리
Message Queue	비동기 메시지 전달	느슨한 결합
Shared Memory	공유 컨텍스트 접근	빠른 데이터 공유
Semaphore	동시 접근 제어	API Rate Limit 관리
Fork-Join	병렬 분산 후 취합	독립 태스크 병렬화

성능 측정 지표

- 태스크 완료 시간 (단일 vs 멀티 에이전트)
- 에이전트 간 통신 오버헤드
- 병렬화 효율 (Speedup Ratio)
- 동기화 대기 시간

4.3 에이전트 데드락 탐지 시스템

아키텍처

```
flowchart LR
    A[멀티 에이전트 시스템] --> B[에이전트 상태 모니터]
    B --> C[Wait-For Graph 빌더]
    C --> D[사이클 탐지 알고리즘]
    D -->|DFS 기반| E{결과}
    E -->|정상| F[정상 계속]
    E -->|데드락| G[회복 전략 선택]
    G --> H1[롤백]
    G --> H2[종료]
    G --> H3[재시작]
```

구현할 데드락 알고리즘

알고리즘	설명	적용 시나리오
Wait-For Graph	대기 관계 그래프 → 사이클 = 데드락	탐지
Banker's Algorithm	안전 상태 검사	예방
Timeout 기반	일정 시간 대기 후 강제 해제	단순 회복
Victim Selection	최소 비용 에이전트 선택 롤백	최적 회복

성능 측정 지표

- 데드락 탐지 시간
- False Positive/Negative 비율
- 회복 성공률
- 롤백 비용 (재처리 토큰 수)

---

5. 1인 프로젝트 성공 전략

5.1 개발 접근법

1. **MVP 우선**: 핵심 기능 2~3개에 집중하여 동작하는 버전 완성
2. **실험 설계 먼저**: 학술적 가치를 위해 비교 실험 설계를 초반에

3. **API 우선 설계**: 프론트엔드는 최소화, 백엔드 로직에 집중
4. **데이터 수집 자동화**: 성능 메트릭, 실험 결과 자동 기록

5.2 교수님 설득 포인트

운영체제 교수님

- "OS 이론(스케줄링, 캐싱, 동기화)을 신규 도메인(LLM 서빙)에 적용"
- "알고리즘별 성능 비교 실험으로 정량적 분석 가능"
- "논문 작성 가능한 주제 (학회/저널 투고)"

설득 스크립트 예시

> "LLM 서비스가 급격히 확산되면서, 다수의 동시 요청을 효율적으로 처리하는 것이 중요한 문제가 되었습니다. 본 프로젝트는 운영체제의 프로세스 스케줄링 알고리즘을 LLM 요청 스케줄링에 적용하여, 처리량과 공정성을 개선하는 방안을 연구합니다. 이는 OS 이론의 실용적 확장이며, 성능 비교 실험을 통해 논문으로 발전시킬 수 있습니다."

5.3 주의사항

항목	설명
API 비용	개발 초기에는 GPT-3.5-Turbo, 실험 시에만 GPT-4 사용
실험 데이터셋	공개 데이터셋 활용 (BEIR, KILT, 한국어-LegalBench)
성능 측정	재현 가능한 실험 환경 구축 (Docker, 고정 시드)
코드 품질	테스트 코드 작성, 문서화 철저히

---

6. 기술 스택 가이드

6.1 기본 스택

```
Runtime: Node.js 20+ / Python 3.11+
Framework: Express.js (API), FastAPI (Python 선택 시)
Database: MongoDB Atlas (+ Vector Search)
Queue: BullMQ (Redis 기반)
실시간: Socket.io
```

6.2 AI 스택

```
LLM API: OpenAI API / Claude API / Gemini API
임베딩: OpenAI Embeddings / HuggingFace Sentence Transformers
RAG: LangChain.js / LlamaIndex
로컬 LLM: Ollama, llama.cpp
NLI 모델: HuggingFace Transformers
```

6.3 실험/분석 도구

```
성능 측정: custom metrics, Prometheus
시각화: Chart.js, Plotly, Matplotlib
통계 분석: Python (pandas, scipy)
```

---

7. 참고 자료

학술 자료

- Efficient Memory Management for Large Language Model Serving - vLLM, PagedAttention
- Orca: A Distributed Serving System for Transformer-Based Generative Models
- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

공식 문서

- MongoDB Atlas Vector Search: https://www.mongodb.com/docs/atlas/atlas-vector-search/
- OpenAI API: https://platform.openai.com/docs
- LangChain.js: https://js.langchain.com

트렌드 리포트

- 2025년 홍익대학교 컴퓨터공학과 졸업 프로젝트 기술 백서
- Gartner Top 10 Tech Trends 2025

---

8. 프로젝트 선택 가이드

```
flowchart LR
    START([시작]) --> Q1{OS 개념을<br/>직접 적용?}
    Q1 -->|YES| Q2{세부 분야?}
    Q2 -->|스케줄링| P1[#1 LLM 추론<br/>스케줄러]
    Q2 -->|IPC/동기화| P2[#2 멀티 에이전트<br/>오케스트레이터]
    Q2 -->|메모리 관리| P3[#3 에이전트<br/>메모리 관리]
    Q2 -->|데드락| P7[#7 에이전트<br/>데드락 탐지]
    Q2 -->|체크포인팅| P9[#9 에이전트<br/>체크포인팅]
    Q1 -->|NO| Q3{자율형 에이전트<br/>관심?}
    Q3 -->|YES| P5[#5 코드 디버깅<br/>또는 #10 자기진화]
    Q3 -->|NO| Q4{보안/신뢰성?}
    Q4 -->|보안| P4[#4 에이전트<br/>샌드박스]
    Q4 -->|합의| P8[#8 분산 에이전트<br/>합의]
    Q4 -->|효율| P6[#6 워크플로우<br/>컴파일러]
```

작성일: 2026년 1월 1일 최종 수정: 2026년 1월 7일 - AI 에이전트 + OS 이론 융합 프로젝트 9개로 전면 개편